



파이썬 입문강의 과제로 개인의 파이썬 프로그램을 완성해야하는데, 최종 완성본은 (1)입출력, 리스트, 조건문 (2)무한 루프와 메뉴 시스템 (반복문) (3)모듈화 및 데이터 확장 (배열과 함수) 를 적용해서 만들어야 함

1차 제출은 프로그램 기획 내용과 이 채팅 내역, 2차 제출은 입출력, 리스트, 조건문을 활용하여 구현, 3차 제출은 무한 루프와 메뉴 시스템 (반복문) 구현, 최종제출은 모듈화 및 데이터 확장 (배열과 함수) 구현 및 프로그램 완성임

1차 과제로 시나리오 기획서와 과정을 제출해야하는데, 지금 내가 생각하는 주제는 테니스 클럽에서 이용할 수 있는 개별 멤버 레이팅 부과, 대회 대진표 생성 시스템이야. 이 주제로 위 조건을 만족하는 프로그램을 만들 수 있을까?

추가로, 이 외에도 프로그래밍을 처음 해보는 대학생에게 추천하는 주제 3가지만 알려줘



네, 생각하신 '**테니스 클럽 레이팅 및 대진표 시스템**'은 교수님이 제시한 모든 조건을 완벽하게 충족할 수 있는 아주 좋은 주제입니다. 데이터의 입력부터 복잡한 로직(대진 매칭)까지 단계별로 확장하기에 최적화되어 있거든요.

이 주제가 어떻게 과제 조건에 연결되는지 정리해 드리고, 프로그래밍 입문자에게 추천하는 다른 주제 3가지도 함께 제안해 드릴게요.

1. 테니스 클럽 시스템의 단계별 구현 로직

(1) 1단계: 입출력, 리스트, 조건문 (초기 구현)

- **기능:** 신규 회원 등록 및 기본 정보 저장.
- **내용:** 회원의 이름, 구력(개월 수), 현재 점수를 입력받아 **리스트**에 저장합니다. **조건문**을 활용해 점수별로 'A조', 'B조' 등급을 자동으로 나누는 기능을 만듭니다.

(2) 2단계: 무한 루프와 메뉴 시스템 (반복문)

- **기능:** 지속적인 데이터 관리.
- **내용:** `while True` 루프를 사용하여 프로그램이 꺼지지 않게 유지합니다. 사용자는 메뉴에서 `1. 회원 추가`, `2. 전체 명단 출력`, `3. 대진표 생성`, `4. 종료` 중 하나를 선택해 작업을 반복할 수 있습니다.

(3) 3단계: 모듈화 및 데이터 확장 (배열과 함수)

- **기능:** 복잡한 로직의 분리 및 대진표 생성.
 - **내용:** 점수를 계산하는 로직(`calc_rating()`), 대진표를 무작위로 섞어 짜는 로직(`make_matchup()`)을 **함수**로 만듭니다. 회원 여러 명의 데이터를 2차원 리스트(배열)로 관리하며 프로그램의 완성도를 높입니다.
-

2. 입문자에게 추천하는 다른 주제 3가지

테니스 주제 외에도 파이썬 입문 단계에서 다루기 좋고, 결과물이 직관적인 주제들입니다.

① 스마트 가계부 및 자산 관리 프로그램

- **설명:** 일일 지출 내역을 입력하고, 카테고리별(식비, 교통비 등)로 합계를 내주는 프로그램입니다.
- **장점:** 숫자를 다루기 때문에 조건문(예산 초과 경고 등)을 넣기 매우 쉽고 실용적입니다.

② 나만의 영화/도서 기록장 (리뷰 시스템)

- **설명:** 내가 본 영화나 책의 제목, 별점, 짧은 평을 저장하고 관리하는 프로그램입니다.
- **장점:** 텍스트 데이터를 리스트로 관리하는 연습을 하기에 좋으며, 별점 기반의 필터링 기능을 넣기 적합합니다.

③ 개인 맞춤형 운동/식단 루틴 생성기

- **설명:** 사용자의 체중과 목적(다이어트, 벌크업)을 입력받아 하루 권장 칼로리를 계산하고, 요일별 운동 루틴 리스트를 보여주는 프로그램입니다.
- **장점:** BMI 계산 등 수학적 공식(함수)을 활용하기 좋고, 성별이나 목적에 따른 다양한 조건문 분기가 가능합니다.

주제는 테니스 클럽 시스템으로 결정할게. 이제 구현해야 하는 기능에 대해 이야기해보자

필요한 기능은 1. 수정 가능한 멤버 리스트 2. 경기 결과를 반영한 레이팅 부여 3. 레이팅에 따른 그룹 분류 4. 그룹 별 대진표 작성
당장 생각나는 건 이정도인데 또 필요한 기능은 뭘까?



테니스 클럽 시스템으로 주제를 확정하셨군요! 말씀하신 4가지 기능은 프로그램의 뼈대를 이루는 핵심 기능들입니다. 여기에 파이썬 입문 과제의 '조건'을 더 잘 충족하면서도 실제 사용성을 높여줄 추가 기능들을 제안해 드릴게요.

대학생 수준의 첫 프로젝트에서 구현하기 적절하면서도 교수님께 좋은 인상을 남길 수 있는 기능들입니다.



테니스 클럽 시스템 추가 제안 기능

1. 경기 이력(Log) 기록 및 조회

단순히 현재 레이팅 점수만 보여주는 것이 아니라, "*"누가 누구와 경기해서 몇 대 몇으로 이겼는지"를 리스트에 쌓아두는 기능입니다.

- **이유:** 리스트(List) 활용 능력을 보여주기 좋고, 나중에 "최근 5경기 결과 보기" 같은 기능을 추가할 때 유용합니다.

2. 회원 검색 및 상세 정보 보기

등록된 멤버가 많아질 경우를 대비해 **이름으로 검색**하여 해당 회원의 승률, 현재 등급, 가입일 등을 출력하는 기능입니다.

- **이유:** 반복문(for)과 조건문(if)을 결합하여 특정 데이터를 찾아내는 로직을 연습하기 좋습니다.

3. 데이터 초기화 및 샘플 데이터 로드

프로그램을 실행할 때마다 매번 회원 10명을 일일이 키보드로 입력하는 건 매우 번거롭습니다. **'테스트용 샘플 데이터 불러오기'** 메뉴를 만들어 미리 준비된 리스트를 변수에 할당해 주는 기능입니다.

- **이유:** 개발 과정이 훨씬 편해지고, 최종 발표 시연 때도 끔찍 없이 보여줄 수 있습니다.

4. 클럽 전체 통계 출력

현재 클럽의 총 인원수, 평균 레이팅 점수, 가장 승률이 높은 '이달의 선수' 등을 한눈에 보여주는 기능입니다.

- **이유:** `len()`, `sum()` 같은 파이썬 내장 함수와 배열 연산을 활용하기에 최적입니다.



단계별 구현을 위한 구조 설계 (기획서용)

제안하신 기능을 포함하여 3단계 과제 제출 일정에 맞춰 구조를 짜보면 다음과 같습니다.

단계

주요 구현 내용

적용되는 파이썬 문법

1단계 (기초)	회원 이름, 구력 입력 및 등급 자동 분류	<code>input()</code> , <code>print()</code> , <code>list</code> , <code>if-else</code>
2단계 (메뉴)	무한 루프 메뉴 (1.등록 2.조회 3.경기입력 4.종료)	<code>while True</code> , <code>break</code> , <code>f-string</code>
3단계 (고급)	레이팅 계산 함수, 대진표 랜덤 생성, 데이터 관리	<code>def</code> (함수), <code>import random</code> , 2차원 리스트

 Sheets로 내보내기



💡 기획서 작성을 위한 팁

1차 과제인 **시나리오 기획서**에는 이 프로그램이 왜 필요한지를 한 문장으로 정리해 보세요.

"본 프로그램은 동호회 운영자가 주관적인 판단 대신 ****객관적인 데이터(레이팅)****를 바탕으로 공정한 대진표를 구성하고, 회원들의 실력 변화를 체계적으로 관리하기 위해 기획되었습니다."

이렇게 적으면 단순한 코딩 과제를 넘어선 '솔루션' 느낌이 납니다. 위 기능들 중에서 특별히 더 구현해보고 싶거나, 로직이 궁금한 부분이 있으신가요?

레이팅 계산에 대해서 구체적으로 정해보자. 이 과제에서는 테니스에서 사용하는 전통적인 방식인 ntrp를 사용하는 것보다 체스에서 사용하는 elo레이팅을 사용하는 것이 더 좋지 않을까? 왜 테니스에서는 ntrp를 사용하는 걸까?



매우 예리한 분석입니다! 입문 프로그래밍 과제 측면에서 보면 **Elo 레이팅**이 훨씬 "코딩할 맛"이 나는 주제인 것은 확실합니다. 왜 테니스 현장에서는 NTRP를 쓰는지, 그리고 왜 과제에서는 Elo가 더 유리한지 비교해 드릴게요.

1. 왜 테니스에서는 NTRP를 더 많이 쓸까?

테니스는 체스와 달리 **신체적 조건**과 **기술의 완성도**가 매우 중요한 종목이기 때문입니다.

- **주관적 평가지표의 필요성:** NTRP(National Tennis Rating Program)는 "포핸드 스트로크를 안정적으로 구사하는가?", "네트 플레이가 가능한가?" 같은 기술적 기준을 기반으로 합니다.
- **직관적인 매칭:** 체스는 점수가 100점 차이 나도 경기가 성립되지만, 테니스는 NTRP 2.5(초보)와 4.5(상급자)가 경기를 하면 랠리 자체가 안 되어 운동이 되지 않습니다. 그래서 처음부터 '급수'를 나눠놓는 형식을 선호합니다.
- **데이터의 부재:** 동호인 수준에서는 모든 경기 결과를 기록 시스템에 입력하기 어렵기 때문에, 한 번 정해지면 오래 유지되는 NTRP 방식이 현장에서 더 편리합니다.

2. NTRP vs Elo 레이팅: 과제 관점에서의 비교

과제 점수를 잘 받기 위해서는 **Elo 방식**이 훨씬 유리합니다. 그 이유를 표로 정리해 보았습니다.

비교 항목	NTRP (전통적 방식)	Elo (체스 방식)
변화 방식	계단식 (고정적)	실시간 (역동적)
주요 문법	단순 <code>if-else</code>	매개변수와 리턴값이 있는 함수 (<code>def</code>)
데이터 활용	등급만 저장	점수의 변동 이력(Log) 저장
프로그래밍 난이도	하 (단순 분류)	중 (수식 계산 및 업데이트)



Sheets로 내보내기



3. 이 과제에서 Elo 레이팅을 추천하는 이유

교수님이 요구하신 **(3) 모듈화 및 데이터 확장** 조건을 충족하기에 Elo 방식이 최적입니다.

- **함수화의 명분:** `update_rating(winner_score, loser_score)` 와 같은 함수를 만들어, 경기 결과

에 따라 리스트 안의 데이터를 실시간으로 수정하는 로직을 보여줄 수 있습니다.

- **알고리즘적 재미:** 나보다 점수가 높은 사람(강자)을 이겼을 때 점수가 더 많이 오르는 로직을 구현하면, 기획서에서 "공정한 레이팅 산출을 위한 알고리즘 적용"이라고 어필하기 좋습니다.

4. 💡 Gemini의 제안: 하이브리드 모델

현실성(테니스)과 과제 점수(코딩)를 모두 잡기 위해 "**Elo로 계산하고, NTRP로 보여주기**" 전략을 추천합니다.

1. **내부 계산:** 모든 회원은 1,000점에서 시작하며, 경기 후 Elo 수식에 따라 점수가 변합니다.
2. **외부 출력:** 사용자에게 보여줄 때는 아래와 같이 출력합니다.

"홍길동 회원님 (현재 점수: 1,250점 / 등급: **NTRP 3.0**)"

간단한 레이팅 함수 예시 (Elo 간소화)

복잡한 지수 승수를 쓰지 않아도 입문 수준에서는 아래 정도의 로직만으로 충분합니다.

$$Score_{new} = Score_{old} + (K \times (Result - Expected))$$

- **Result:** 승리 시 1, 패배 시 0
- **Expected:** 상대와의 점수 차이에 따른 예상 승률 (입문 단계에서는 0.5 등으로 고정하거나 간단한 차이값으로 대체 가능)

결론: Elo 레이팅을 채택하시겠어요?

Elo 방식을 선택하신다면, 1차 기획서에 *****단순 등급제를 넘어 경기 결과에 따른 실시간 레이팅 업데이트 시스템을 구현하겠다*****라고 적으시면 됩니다.

이 방식으로 확정하신다면, 다음 단계로 **'대진표 생성'** 시 레이팅을 어떻게 활용할지(예: 비슷한 점수끼리 붙이기 vs 랜덤) 고민해 보면 좋을 것 같습니다. 어떤 로직이 더 끌리시나요?



테니스 클럽내에서 현실적으로 사용하려면 ntrp보다는 elo레이팅이 훨씬 유용할 것 같으니 elo레이팅을 사용하는 것으로 하자
하이브리드 방식을 채택할지 일반적인 elo레이팅을 채택할지는 나중에 정하자



좋은 결정입니다! **Elo 레이팅**을 선택하면 단순한 관리를 넘어 '데이터 분석'의 묘미가 생기기 때문에 과제 완성도 면에서 훨씬 유리할 거예요.

이제 1차 과제인 **시나리오 기획서** 작성을 위해 전체적인 프로그램 구조를 설계해 봅시다. 이 내용은 그대로 복사해서 과제물에 활용하시거나 참고하시면 됩니다.

[프로그램 기획서] 테니스 클럽 실시간 레이팅 및 대진 시스템

1. 프로그램 개요

- **배경:** 동호인 테니스 클럽에서 주관적인 실력 판단으로 인한 대진 불균형을 해소하고자 함.
- **목적:** 경기 결과를 바탕으로 실시간으로 변동되는 **Elo 레이팅 시스템**을 도입하여 객관적인 실력 지표를 제공하고, 이를 기반으로 공정한 대진표를 자동 생성함.

2. 주요 기능 정의 (단계별 구현 계획)

[1차 구현] 기초 데이터 및 관리 (입출력, 리스트, 조건문)

- **멤버 등록:** 이름, 초기 레이팅(기본 1000점), 선호 시간대 등을 입력받아 리스트에 저장.
- **멤버 수정/삭제:** 등록된 멤버의 정보를 변경하거나 탈퇴 처리.
- **자동 등급 부여:** 레이팅 점수에 따라 'Beginner', 'Intermediate', 'Advanced' 등 등급(Condition) 출력.

[2차 구현] 메뉴 시스템 및 반복 제어 (무한 루프)

- **메인 메뉴 구축:** `while` 루프를 통해 사용자가 '종료'를 선택하기 전까지 지속적으로 서비스 제공.

1. 멤버 관리 | 2. 경기 결과 입력 | 3. 레이팅 랭킹 보기 | 4. 대진표 생성 | 5. 종료

- **예외 처리:** 메뉴 외 숫자를 입력했을 때 안내 메시지 출력.

[3차 구현] 레이팅 알고리즘 및 대진 자동화 (함수, 배열 확장)

- **Elo 계산 함수:** `calc_elo(winner_rating, loser_rating)` 함수를 구현하여 승패에 따른 점수 이동 처리.
 - **대진표 생성 알고리즘:** * **랜덤 방식:** 전체 참여 인원을 무작위로 섞어 매칭.
 - **밸런스 방식 (심화):** 레이팅이 비슷한 사람들끼리 인접하게 매칭하여 경기 질 향상.
 - **데이터 모듈화:** 데이터 처리 로직과 화면 출력 로직을 별도의 함수로 분리하여 코드 가독성 증대.
-

3. Elo 레이팅 로직 설계 (미리 보기)

과제에서 구현할 핵심 수식의 논리는 다음과 같습니다. 복잡한 공식 대신 입문 수준에서 이해하기 쉬운 **간소화된 알고리즘**을 적용할 예정입니다.

1. **기본 점수:** 모든 회원은 1000점에서 시작합니다.
2. **가중치(K-Factor):** 한 경기당 최대 변동 폭을 결정합니다 (예: 32점).
3. **예상 승률:** 본인 점수와 상대 점수의 차이를 이용해 누가 이길 확률이 높은지 계산합니다.
4. **점수 업데이트:** * 약자가 강자를 이기면 점수가 크게 상승합니다.
 - 강자가 약자를 이기면 점수가 소폭 상승합니다.



파이썬 입문 과제에서 Elo 레이팅의 **간소화된 알고리즘**을 사용해야 하는 이유는 크게 세 가지입니다. 이는 단순히 계산을 쉽게 하려는 목적을 넘어, ****코딩 교육의 목적****과 **'프로그램의 안정성'** 때문입니다.

1. 수학적 복잡도와 파이썬 라이브러리 활용의 한계

원래의 Elo 레이팅 수식은 지수 함수(10의 거듭제곱)를 포함합니다.

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

- **문제점:** 입문 단계에서는 `math` 모듈을 불러와서 거듭제곱(`**`)이나 복수 연산을 처리하는 과정에서 **부동 소수점(Floating point)** 오류가 발생하기 쉽고, 코드가 지나치게 복잡해집니다.
- **해결책:** "상대와의 점수 차이"를 0~1 사이의 상수로 단순화하여 계산하면, 순수하게 **리스트 제어와 기본 연산**에만 집중할 수 있습니다.

2. '함수(Function)'의 역할 강조

교수님이 요구하신 (3)번 조건인 **모듈화**를 충족하려면, 함수가 명확한 '입력'과 '출력'을 가져야 합니다.

- **간소화 알고리즘:** `def update_rating(winner_score, loser_score):` 처럼 인자를 받아 간단한 덧셈/뺄셈 후 결과를 돌려주는 구조가 됩니다.

- **이점:** 코드를 처음 보는 사람(또는 채점자)이 함수의 로직을 한눈에 파악할 수 있어 **가독성 점수**에서 유리합니다.

3. 데이터 변화의 가시성 (테스트 용이성)

정밀한 수식을 쓰면 점수가 1.234... 단위로 미세하게 변할 수 있습니다.

- **간소화 알고리즘:** 승리 시 +30, 패배 시 -20 처럼 **정수 단위**로 확실하게 변하게 설계하면, 프로그램이 잘 돌아가는지 확인(디버깅)하기가 훨씬 쉽습니다.
- **효과:** 리스트에 담긴 데이터가 바뀌는 것을 눈으로 확인해야 하는 입문자에게는 정수 기반의 큰 변화폭이 더 적절합니다.

💡 기획서에 적을 '간소화 알고리즘' 논리 예시

기획서의 '알고리즘 설계' 부분에 다음과 같이 적으시면 교수님께서 "현명한 판단을 했다"고 생각하실 겁니다.

"본 프로그램은 입문 단계의 데이터 처리에 최적화하기 위해, 복잡한 지수 연산 대신 점수 차이에 따른 가중치 합산 방식의 '간소화된 Elo 알고리즘'을 채택함. 이를 통해 계산 오차를 줄이고 함수의 가독성을 높여 유지보수가 용이하도록 설계함."

추천하는 간소화 방식 (코드 로직)

1. 기본 변동치(K): 30점
2. **업셋(Upset) 가산점:** 만약 점수가 낮은 사람이 높은 사람을 이겼다면 $K + 10$ 점을 부여하고, 반대라면

K - 10 점만 부여.

3. **결과:** 이 간단한 `if` 문 하나만으로도 Elo 레이팅의 핵심인 '강자를 이겼을 때의 보상'을 충분히 구현할 수 있습니다.

이 정도의 논리면 1차 기획서에서 알고리즘 부분은 완벽하게 방어가 될 것 같습니다. 이제 이 로직을 바탕으로 **메뉴 구성(1번은 무엇, 2번은 무엇...)**을 확정해 볼까요?

제출 차수별로 구현해야하는, 또는 구현할 수 있는 기능에 대해 구체적으로 이야기해보자



교수님이 제시한 3단계 제출 일정에 맞춰, 테니스 클럽 시스템의 기능을 '**학습 진도**'와 '**코드의 확장성**'에 따라 구체적으로 배분해 보았습니다.

각 차수에서 배운 문법을 가장 잘 보여줄 수 있는 기능들로 구성했습니다.



테니스 클럽 시스템 단계별 로드맵

1차 제출: 시나리오 기획서 (현재 단계)

- **할 일:** 프로그램의 목적, 대상 사용자, 주요 기능 목록, 데이터 구조(리스트 구성), 레이팅 알고리즘(간소화

된 Elo)의 논리적 설계를 문서로 정리합니다.

- **포인트:** "왜 이 프로그램이 필요한가?"와 "어떻게 구현할 것인가?"를 명확히 기술합니다.

2차 제출: 입출력, 리스트, 조건문 (기초 구현)

파이썬의 가장 기초적인 데이터를 다루는 단계입니다. *****단일 사용자 등록과 판정*****에 집중합니다.

- **구현 기능:**
 1. **회원 정보 입력:** 이름, 구력(개월), 현재 점수(기본 1000점)를 개별 변수나 리스트에 저장.
 2. **NTRP 등급 판정:** `if-elif` 문을 사용하여 입력된 점수에 따라 "당신은 NTRP 3.0입니다"라고 출력.
 3. **간단한 점수 수정:** 경기 결과를 '승' 혹은 '패'로 입력받아 점수를 30점 더하거나 빼는 기초 로직.
- **핵심 코드 구조:**

Python



```
member_name = input("이름: ")
score = 1000
if score >= 1200:
    grade = "3.5"
```

3차 제출: 무한 루프와 메뉴 시스템 (반복문 구현)

이제 프로그램이 한 번 실행되고 끝나는 게 아니라, **관리 도구**로서의 형태를 갖추는 단계입니다.

- 구현 기능:

- 메인 메뉴 화면:** `while True` 를 사용하여 사용자가 '종료'를 누르기 전까지 메뉴가 계속 반복 출력 됨.
- 다수 회원 관리:** 리스트 안에 리스트(2차원 리스트)를 사용하여 여러 명의 회원을 저장.
- 목록 전체 출력:** `for` 문을 사용하여 현재 등록된 모든 회원의 이름과 점수를 표 형식으로 출력.
- 조건부 검색:** 특정 점수 이상인 회원만 필터링해서 보기.

- 메뉴 구성 예시:

- 회원 등록 | 2. 전체 회원 조회 | 3. 점수 수정 | 4. 종료

최종 제출: 모듈화 및 데이터 확장 (배열과 함수 완성)

가장 어려운 로직을 **함수**로 분리하여 코드의 완성도를 높이고, 대망의 **대진표 생성**을 추가하는 단계입니다.

- 구현 기능:

- Elo 레이팅 함수화:** 점수 계산 로직을 `def update_rating(winner, loser)` 함수로 만들어 코드 중복 제거.

2. **자동 대진표 생성:** `import random` 을 사용하여 리스트에 있는 인원들을 2명씩 짝지어 출력.
3. **데이터 정렬:** `sort()` 기능을 활용해 레이팅이 높은 순서대로 랭킹 보드 출력.
4. **정보 수정/삭제:** 특정 회원을 이름으로 찾아 정보를 수정하거나 삭제하는 완전한 데이터 관리 (CRUD).

- **최종 완성 포인트:**

- 함수를 사용해 메인 코드가 간결해야 함.
 - 2차원 배열(리스트 내 리스트)을 자유자재로 컨트롤하는 모습 제시.
-

구현 시 주의할 점 (꿀팁)

1. **데이터의 영속성:** 아직 파일 저장(DB)은 배우지 않았으므로, 프로그램이 꺼지면 데이터가 날아갑니다. 1차 기획서에 *****현재 버전은 메모리 기반으로 작동하며, 추후 파일 입출력으로 확장 가능함*****이라고 적어두면 좋습니다.
2. **예외 처리:** 메뉴 선택 시 숫자가 아닌 문자를 넣었을 때 프로그램이 꺼지지 않도록 간단한 `if` 처리를 해주는 것이 점수에 유리합니다.
3. **샘플 데이터:** 최종 제출 시, 교수님이 바로 확인하실 수 있도록 미리 회원 5~6명이 등록된 상태로 시작하게 코드를 짜는 것이 시연에 좋습니다.

다수 회원 관리: 리스트 안에 리스트(2차원 리스트)를 사용하여 여러 명의 회원을 저장.
조금 더 구체화해줘



파이썬에서 **'**2차원 리스트'** 를 사용하여 다수의 회원을 관리한다는 것은, 쉽게 말해 **엑셀 시트 하나를 통째로 파이썬 변수에 담는 것과 같습니다.**

입문 단계에서 가장 많이 쓰이는 이 구조를 테니스 클럽 시스템에 어떻게 적용할지 데이터 구조와 활용 예시를 통해 구체화해 드릴게요.

1. 데이터 구조 설계 (엑셀 형태)

리스트 안에 또 다른 리스트가 들어가는 구조입니다. 각 내부 리스트는 '한 명의 회원 정보'를 담는 행(Row) 역할을 합니다.

인덱스	이름 (0)	현재 레이팅 (1)	구력 (2)
0	"김테니스"	1250	12
1	"이서브"	1100	6
2	"박발리"	1450	24

2. 파이썬 코드로 표현하기

이 데이터는 파이썬에서 다음과 같이 선언되고 관리됩니다.

Python



```
# 전체 회원 명단 (2차원 리스트)
members = [
    ["김테니스", 1250, 12, "Intermediate"],
    ["이서브", 1100, 6, "Beginner"],
    ["박발리", 1450, 24, "Advanced"]
]
```

3. 기능별 구체적 활용법 (코드 로직)

① 특정 데이터에 접근하기 (인덱싱)

- `members[0]` : 첫 번째 회원인 "김테니스"의 전체 정보 리스트를 가져옵니다.
- `members[1][0]` : 두 번째 회원의 '이름'("이서브")만 가져옵니다.
- `members[2][1]` : 세 번째 회원의 '레이팅 점수'(1450)만 가져옵니다.

② 새로운 회원 추가하기 (`append`)

사용자에게 입력을 받아 새로운 리스트를 만들고, 이를 `members` 라는 큰 리스트에 추가합니다.

Python



```
new_name = input("이름 입력: ")
new_score = 1000
new_info = [new_name, new_score, 0, "Beginner"]
members.append(new_info) # 리스트 안에 리스트가 추가됨
```

③ 반복문을 이용한 전체 명단 출력 (`for`)

2차원 리스트의 진가는 여기서 나타납니다. `for` 문을 돌리면 회원 한 명씩 정보를 꺼내어 예쁘게 출력할 수 있습니다.

Python



```
print("--- 클럽 회원 명단 ---")
for m in members:
    # m은 ["이름", 점수, 구력, 등급] 형태의 리스트가 됨
    print(f"이름: {m[0]} | 점수: {m[1]} | 등급: {m[3]}")
```

④ 데이터 수정 (경기 결과 반영)

경기가 끝나서 "김테니스"의 점수를 올리고 싶다면 해당 위치의 값을 직접 수정합니다.

Python



```
# 김테니스의 점수(인덱스 1번)를 30점 증가  
members[0][1] += 30
```

4. 과제 제출 시 '데이터 확장'의 의미

교수님이 언급하신 '**데이터 확장**'은 바로 이 2차원 리스트 구조를 의미합니다.

- **확장 전:** 변수 하나에 회원 한 명 이름만 저장 (`name1 = "김철수"`) -> 회원이 늘어날수록 변수를 무한정 만들어야 함.
- **확장 후:** `members` 라는 리스트 하나에 수천 명의 회원을 담을 수 있고, `for` 문 하나로 모든 회원을 관리할 수 있음.

그러면 지금까지의 기획 내용을 제출 차수 별로 최종정리를 해서 한번 더 보여줘



테니스 클럽 레이팅 및 대진 시스템의 **1차 기획서 제출용 최종 정리본**입니다. 이 내용을 바탕으로 기획서를 작성하시면 교수님이 요구한 모든 기술적 조건과 실무적 논리가 완벽하게 담길 것입니다.



[최종 기획안] 테니스 클럽 실시간 레이팅 및 대진 관리 시스템

1. 프로그램 개요

- **목적:** 동호회 운영자가 회원들의 실력을 **Elo 레이팅** 기반으로 객관적으로 관리하고, 이를 바탕으로 실력이 비슷한 회원들끼리 **자동 대진**을 생성하여 경기 만족도를 높이는 솔루션 제공.
 - **핵심 가치:** 주관적인 등급 분류에서 벗어나 경기 결과에 따른 역동적인 데이터 관리.
-

2. 차수별 구현 상세 로드맵

[1차] 시나리오 및 알고리즘 설계 (현재)

- **핵심 로직:** 간소화된 Elo 레이팅 수식 설계.
 - 기본 K-지수(변동폭)를 설정하고, 승패에 따라 점수를 가감.
 - 강자를 이겼을 때 더 높은 점수를 얻는 '업셋(Upset)' 로직 반영.

[2차] 기초 구현 (입출력, 리스트, 조건문)

- **목표:** 단일 유저 데이터 처리 및 판정 로직 완성.
- **기능:** * `input()` 으로 회원 이름, 구력, 초기 점수 입력.
 - `if-elif` 조건문을 사용하여 점수대별 **NTRP 등급(2.5, 3.0, 3.5 등)** 자동 매칭 후 출력.
 - 기초적인 승/패 입력에 따른 점수 합산 테스트.

[3차] 시스템 고도화 (무한 루프와 메뉴 시스템)

- **목표:** 다수 회원 관리 및 지속적인 프로그램 운영 환경 구축.
- **기능:** * `while True` 기반의 **메인 메뉴** 구성 (1. 등록, 2. 조회, 3. 경기입력, 4. 종료).
 - **2차원 리스트(Nested List)** 도입: `members = [ ["이름", 점수, 구력, "등급"], ... ]` 구조로 데이터 확장.
 - `for` 문을 활용한 전체 회원 명단 및 랭킹 순위표 출력.

[최종] 모듈화 및 기능 완성 (배열과 함수)

- **목표:** 코드의 재사용성을 높이고 핵심 기능(대진표) 완성.
 - **기능: * 함수화:** `add_member()`, `update_rating()`, `make_matchup()` 등 주요 기능을 독립된 함수로 분리.
 - **자동 대진 생성:** `import random` 을 활용하여 레이팅이 유사한 회원들끼리 대진표를 짜주는 알고리즘 구현.
 - **데이터 검색/삭제:** 특정 회원을 이름으로 찾아 정보를 수정하거나 명단에서 삭제하는 기능.
-

3. 데이터 구조 상세 (2차원 리스트)

프로그램 내부에서 회원은 다음과 같은 배열 형태로 관리됩니다.

- **구조:** `members[회원번호][데이터인덱스]`
 - `members[n][0]` : 회원 이름 (String)
 - `members[n][1]` : Elo 레이팅 점수 (Integer)
 - `members[n][2]` : 구력/개월수 (Integer)
 - `members[n][3]` : 계산된 NTRP 등급 (String)
-

4. 기대 효과 및 확장성

- **정확성:** 수치화된 점수를 통해 회원 간 실력 차이를 명확히 파악.
- **공정성:** 랜덤 혹은 레이팅 기반 대전 생성을 통해 운영자의 주관 개입 배제.
- **확장성:** 추후 파일 입출력을 통해 데이터를 저장하거나, 웹 인터페이스와 연동 가능.